

1 Calculabilité

1.1 Rappels

Automate : modèle mathématique de calcul.

Automate fini déterministe (AFD) : automate avec un seul état possible à chaque instant.

Automate fini non déterministe (AFN) : automate avec plusieurs états possibles à chaque instant.

Alphabet : ensemble fini t.q. $\Sigma = \{0, 1\}$

Mot : suite finie de symboles d'un alphabet. Ex : $w = 0110 \in \Sigma^4$

Langage : ensemble de mots sur un alphabet. Ex : $L = \{w | w \text{ commence par } 0\}$

Taille : nombre de symboles d'un mot. Ex : $|w| = 4$

Concatenation : opération binaire entre deux mots. Ex : $w_1 = 01, w_2 = 10 \Rightarrow w_1 w_2 = 0110$

Concatenation : opération binaire entre deux mots. Ex : $w_1 = 01, w_2 = 10 \Rightarrow w_1 w_2 = 0110$

Ensemble des langages : ensemble des parties de Σ^* . Ex : $\mathcal{L} = \mathcal{P}(\Sigma^*)$

Renversé : mot lu à l'envers. Ex : $w = 0110 \Rightarrow w^R = 0110$

1.2 Machines de Turing

Définition : une machine de Turing est un 7-uplet $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ où :

- Q est un ensemble fini d'états
- Σ est un alphabet d'entrée ne contenant pas le symbole blanc $\sqcup$
- Σ^* est l'ensemble de tous les mots sur Σ
- Γ est un alphabet de bande, avec $\Sigma \subset \Gamma$
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{G, D\}$ est la fonction de transition
- $q_0 \in Q$ est l'état initial
- $q_{\text{accept}} \in Q$ est l'état d'acceptation
- $q_{\text{reject}} \in Q$ est l'état de rejet, avec $q_{\text{reject}} \neq q_{\text{accept}}$ (jamais dessiné, mais tjrs là)
- Tête lecture : tjrs sur la bande, peut lire et écrire des symboles, et se déplacer à gauche (L) ou à droite (R)

Language reconnu : on dit que la MT M reconnaît un langage L si pour tout mot $w \in \Sigma^*$, M accepte w si et seulement si $w \in L$. (aussi appelé problème)

MT à k rubans : une MT à k rubans est une MT avec k bandes, chacune avec sa propre tête de lecture.

- On les note $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R\}^k$

- Émulable par une MT à un seul ruban en temps poly.

MT bi-dimensionnelle : une MT avec un nombre infini de rubans est émulable par une MT à un seul ruban en temps poly.

NOTE : Les objets utilisés dans le pseudo-code sont émulable en temps poly par une MT à un seul ruban.

2 Décidabilité

2.1 Définitions

Décideur : M MT est décideur de L si M s'arrête sur toute entrée et $M(w) \iff w \in L$

Reconnaisseur : M MT est un reconnaisseur de L si $M(w) = 1 \iff w \in L$

CO-REC : un langage est dans CO-REC s'il existe une MT qui reconnaît son complément. (Si $\text{REC} \cap \text{CO-REC}$, alors DEC)

Language régulier (REG) : un langage est régulier s'il peut être reconnu par un automate fini.

Language décidable (DEC) : un langage est décidable s'il peut être reconnu par une machine de Turing qui s'arrête pour tout mot d'entrée.

Language reconnaissable (REC) : un langage est reconnaissable s'il peut être reconnu par une machine de Turing qui s'arrête pour les mots du langage, mais peut ne pas s'arrêter pour les mots qui n'appartiennent pas au langage.

- NOTE : Ne pas oublier les compléments de ces langages, qui peuvent être indécidables ou non reconnaissables.

Théorème : $\text{REG} \subsetneq \text{DEC} \subsetneq \text{REC} \subsetneq \mathcal{L}$

Théorème : $A \leq_m B$ et B est DEC $\Rightarrow A$ est DEC et $A \leq_m B \wedge B \in \text{REC} \rightarrow A \in \text{REC}$.

Corrolaire : $A \leq_m B$ et A est non DEC $\Rightarrow B$ est non DEC et $A \leq_m B \wedge A \notin \text{REC} \rightarrow B \notin \text{REC}$.

Théorème de Rice : tout langage non trivial (c'est-à-dire, tout langage qui n'est ni vide ni égal à Σ^*) est non DEC.

Machines intéressantes :

- A_{AFD} : langage des mots acceptés par un AFD.

Props : A_{AFD} est REG, DEC et REC.

- A_{MT} : langage des mots acceptés par une MT.

Props : A_{MT} est REC, mais non DEC.

- $HALT_{MT}$: langage des mots pour lesquels une MT s'arrête.

Props : $HALT_{MT}$ est REC, mais non DEC.

- $SELF$: langage des mots qui représentent une

MT qui s'accepte elle-même.

Props : $SELF$ est REC, mais non DEC.

- E_{MT} : langage des mots pour lesquels une MT ne s'arrête pas.

Props : E_{MT} est REC, mais non DEC.

2.2 Réduction

Définition : une réduction de A à B est une fonction calculable $f : \Sigma^* \rightarrow \Sigma^*$ t.q $\forall w, w \in A \iff f(w) \in B$.

Théorème de réduction : si $A \leq B$ et B est DEC, alors A est DEC.

Exemple de réduction : $A_{MT} \leq HALT_{MT}$

- Construction de f : $f(w) = \langle M_w, w \rangle$ où M_w est une MT qui simule M sur w et s'arrête si M accepte w .

- Preuve : $w \in A_{MT} \iff M \text{ accepte } w \iff M_w \text{ s'arrête sur } w \iff f(w) \in HALT_{MT}$

Language multivoque-réductible : un langage A est multivoque-réductible à un langage B s'il existe une fonction calculable $f : \Sigma^* \rightarrow \Sigma^*$ t.q $\forall w, w \in A \iff f(w) \in B$ et f peut produire plusieurs images pour un même mot d'entrée.

Turing-réductible : un langage A est Turing-réductible à un langage B s'il existe une machine de Turing qui peut décider A en utilisant un oracle pour B .

MT minimale : une MT minimale pour un langage L est une MT qui reconnaît L et a le moins d'états possible.

Turing-reconnaissable : C'est équivalent à la notion de langage reconnaissable (REC).

2.3 Trucs supp.

- $\leq_m$: notation pour la réduction multivoque-réductible.
- $\leq_T$: notation pour la réduction Turing-réductible.
- $\overline{A_{MT}}$ n'est pas REC, alors :
 - $L \in DEC \iff L \in REC \wedge \overline{L} \in REC$.
 - Si $A \leq_m B \wedge A \notin REC/DEC$, alors $B \notin REC/DEC$.

2.4 Théorème de la récursion

Théorème de la récursion : pour toute machine de Turing M et tout mot w , il existe une machine de Turing M' telle que M' accepte un mot x si et seulement si M accepte le mot w .

Preuve : on peut construire M' de la manière suivante :

- - M' ignore son entrée x et simule M sur w .
- Si M accepte w , alors M' accepte x .
- Si M rejette w , alors M' rejette x .
- Si M ne s'arrête pas sur w , alors M' ne s'arrête pas sur x .

3 P vs NP

Classe P : classe des langages qui peuvent être décidés par une machine de Turing déterministe en temps polynomial.

Classe NP : classe des langages qui peuvent être décidés par une machine de Turing non déterministe en temps polynomial.

Problème NP-complet : un problème est NP-complet s'il est dans NP et que tout problème dans NP peut être réduit à lui en temps polynomial.

Problème NP-difficile : un problème est NP-difficile s'il est au moins aussi difficile que les problèmes NP-complets, c'est-à-dire que tout problème NP-complet peut être réduit à lui en temps polynomial.

Certificat : un certificat pour un problème de décision est une preuve qui peut être vérifiée en temps polynomial par une machine de Turing déterministe.

Vérificateur : un vérificateur pour un problème de décision est une machine de Turing déterministe qui prend en entrée un mot et un certificat, et qui accepte si le certificat est valide pour le mot.

Réduction polynomiale : une réduction de A à B est une fonction calculable en temps polynomial $f : \Sigma^* \rightarrow \Sigma^*$ t.q $\forall w, w \in A \iff f(w) \in B$. (Noté $A \leq_p B$)

Exemples de problèmes NP-complets :

- **SAT** : problème de satisfiabilité des formules booléennes.
- **3-SAT** : problème de satisfiabilité des formules

booléennes en forme normale conjonctive avec au plus 3 littéraux par clause.

- **Clique** : problème de trouver une clique de taille k dans un graphe.

- **Vertex Cover** : problème de trouver un ensemble de sommets de taille k qui couvre toutes les arêtes d'un graphe.

- **Subset Sum** : problème de trouver un sous-ensemble d'un ensemble de nombres qui somme à un nombre donné.

Set-cover : problème de trouver un sous-ensemble de sous-ensembles qui couvre un ensemble universel.

Théorème de Cook-Levin : SAT est NP-complet.

Théorème de Karp : 3-SAT, Clique, Vertex Cover et Subset Sum sont NP-complets.

- TRUC : On choisit 3-SAT comme réduction lorsque le problème est structurellement local (ex. : 2 ou 3 objets, graphes, etc.) et on prend SAT lorsque le problème est plus général ou plus complexe (ex. : circuits logiques, problèmes avec contraintes arbitraires).

4 Extra pour l'exam

Étapes pour dessiner une MT :

- Identifier les états nécessaires pour le processus de calcul.
- Définir les transitions entre les états en fonction des symboles lus et des actions à effectuer (écrire, déplacer la tête).
- S'assurer que la machine a un état d'acceptation.
- Vérifier que la machine est complète, c'est-à-dire qu'elle a une transition définie pour chaque combinaison d'état et de symbole lu.

Pour calculer le tmp d'exéc. d'une MT :

- Compter le nombre de transitions effectuées par la machine sur un mot d'entrée donné.
- Chaque transition prend un temps constant.
- Le temps total est proportionnel au nombre de transitions effectuées.

Implication entre langage et réduction : Voici les étapes pour trouver si une propriété d'un langage s'applique aussi à un langage à partir duquel il est réductible :

- Identifier la propriété que vous souhaitez vérifier pour le langage cible (par exemple, si le langage est décidable ou reconnaissable).
- Trouver un langage de référence pour lequel la propriété est connue (par exemple, A_{MT} pour les langages non décidable).
- Vérifier si le langage cible est réductible au langage de référence en utilisant une réduction appropriée (par exemple, une réduction multivoque-réductible ou une réduction Turing-réductible).
- Si la réduction est réussie et que le langage de référence possède la propriété en question, alors le langage cible possède également cette propriété.

Pour montrer qu'un problème est NP-complet :

- Montrer que le problème est dans NP.
- Choisir un problème NP-complet connu.
- Réduire le problème choisi au problème en question.

(Se référer au truc pour choisir entre 3-SAT et SAT pour la réduction)